

Asymptotic Analysis: An Incremental Contract Audit

A Mathics predicate-boundary risk, exact signed Lerch enclosures,
and differentiable Zeta tails

Technical review prepared for Vladimir Reshetnikov

10 September 2026

Reviewed revision: 8cee870994f506b501bae3ea6bd4a3a7edb895c1

Abstract

This review examines a pinned revision of the Asymptotic repository while treating its existing code-review register, intake records, retained reports and retirement records as an exclusion baseline. It does not repackage the already catalogued correctness, performance, validation and roadmap issues as fresh findings.

The incremental result is deliberately differentiated: one source-level compatibility risk requiring native confirmation, and two concrete, proved provider extensions. The risk concerns protection of unevaluated real-domain predicates in an inline `ConditionalExpression` on the Mathics package path. The extensions are a mean–variance enclosure for the signed large-argument Lerch remainder, and an all-orders differentiability contract for the admitted Zeta Dirichlet tail. An independent exact-rational implementation of the Lerch enclosure passed 576 parameter/order cases against independently bounded defining-series sums. Seven Python test groups passed. The supplied Wolfram Language implementation and native package probes were not executed.

No Wolfram 15 or Mathics kernel was available for package execution in this environment; the attempted Wolfram evaluator connection failed. Accordingly, this article is not a dual-runtime certification, and it does not claim a reproduced new wrong-result bug. It supplies the source paths, mathematical arguments, acceptance tests, limitations and implementation boundaries needed to decide and integrate the incremental work.

Reading guide. Section 3 contains the compatibility risk and proposed repair. Section 4 contains the executed reference implementation’s mathematical basis. Section 5 proves the derivative extension. Sections 6–7 describe validation and integration. The archive contains this source, the PDF, code, probes and evidence records; it contains no checksum files.

Contents

1	Scope, evidence and incremental verdict	3
2	Inspection map and selection of the new work	4
3	N1: a source-predicate protection gap in Mathics	5
4	E1: a signed mean–variance enclosure for the Lerch remainder	8
5	E2: differentiable Zeta Dirichlet tails	12
6	Validation, portability and artifact status	15
7	Integration priorities and development recommendations	16
8	Conclusion	17
A	Running the supplied artifacts	17
B	Artifact inventory	18

1 Scope, evidence and incremental verdict

1.1 The snapshot and the exclusion rule

The subject is [VladimirReshetnikov/Asymptotic](#), at the revision stated on the title page. References to repository behavior in this article are to that revision, not to a moving `main`. The tree was accessed through the GitHub connector. A complete local clone was not obtained; selected source files and contiguous ranges were inspected through repository reads.

The exclusion baseline was assembled from the code-review root index, the central status register, wave intake records, and the wave indexes. Those records identify 45 numbered report packages, of which 35 are retained and ten are represented by retirement records. Retirement is not treated as permission to rediscover a previously reported mechanism. In particular, a report moved out of its former directory remains part of the exclusion history. The report paths and the review method are documented in the archive’s novelty crosswalk.[1, 2, 3, 4, 5]

This was not a line-by-line reading of every retained historical article and every retired article’s original bytes. The maintained registers and intake crosswalks supplied the principal mechanism-level deduplication evidence. Thus “incremental” here means that no matching mechanism or the specific construction below was identified in that consulted baseline; it is not a mathematical guarantee that no sentence in any historical report anticipates it.

The article excludes familiar repository-wide recommendations already present in that baseline. It also distinguishes a genuinely new mechanism from a new example of an old mechanism.

1.2 Evidence classes

Source-established means that a statement follows from the inspected implementation, such as which syntax positions a protection routine visits or the derivative-order value that a constructor stores. **Mathematically established** means that an explicit argument is given in the article. **Executed reference check** means that the independent Python implementation was run in this environment. **Native characterization pending** means that the supplied Wolfram/Mathics probe was not run; no imagined output is substituted for it.

These classes are intentionally not interchangeable. A source guard can demonstrably omit a position without the complete public call necessarily returning a wrong answer: later validation may refuse the call. A theorem about an infinite tail does not establish that a particular package representation preserves its fields. An exact Python enclosure does not establish the behavior of a Wolfram Language translation.

1.3 What this review can and cannot conclude

The inspected source supports a substantial separation between an analytic package result, native delegation, branch information, remainder metadata and numerical checks. The main package advertises real power–log expansions and inverse branches, specialized Gamma/Barnes scales and explicit native delegation. The source also distinguishes native results from package analytic contracts. These are relevant architectural facts, not evidence that every route is correct.[6, 7]

The available evidence does not support a statement that the entire repository passes on either target kernel, a comparative timing claim, a new release-readiness verdict, or a count of newly reproduced runtime bugs. The strongest deliverables here are narrower: a precise unprotected syntax boundary, an independently tested exact enclosure, and a proof that justifies an automatic derivative capability on a specific existing provider.

ID	Incremental result	Status and immediate action
N1	Inline source predicates lie outside the Mathics input-option protection boundary.	Source omission established; public wrong-result manifestation unexecuted. Prioritize native characterization and a source-aware repair if confirmed.
E1	Signed Lerch remainder enclosure using a size-biased geometric variable and a beta variable.	Proved for positive real parameters; exact-rational positive-integer implementation executed. Integrate as a restricted provider enhancement.
E2	All-orders derivative bounds for the Zeta Dirichlet tail in its exponential coordinate.	Proved, with an explicit finite bound. The current default contract is conservative; native integration and output tests remain pending.

Table 1: The incremental inventory. E1 and E2 are enhancements, not allegations that the current numerical bounds are false.

2 Inspection map and selection of the new work

The review followed contracts across call boundaries rather than assuming that similarly named operations share the same semantics. The inspected paths included held public dispatch, assumption separation, exact special-function reductions, asymptotic coefficient construction, arithmetic and derivative representations, inverse source coordinates, numerical certificates and selected build scripts. Table 2 records the substantive areas. It is a map of inspected evidence, not a claim that every file in each area was read in full.

Table 2: Selected source inspection and its role in this review.

Area	Files or routines inspected	Result used here
Public request handling	<code>AsymptoticAnalysis.wl</code> ; <code>NativeCompatibility.wl</code> ; <code>InverseFunctionSyntax.wl</code> ; <code>InverseFunctionExpressions.wl</code>	Located the held-to-evaluated transition and subsequent extraction of source conditions. Basis for N1.
Mathics assumptions	<code>MathicsAssumptions.wl</code> ; <code>MathicsInputAssumptions.wl</code> ; compatibility and Taylor adapters	Verified that the additional input protection is keyed to syntactic assumption-option values. Basis for N1.
Dirichlet providers	<code>DirichletSpecialFunctions.wl</code>	Checked the defining sums, first-omitted indices, current absolute Lerch bound, and finite-tail derivative flag. Basis for E1 and E2.
Exact special-function reductions	<code>SpecialFunctionIdentities.wl</code> ; <code>ParameterizedSpecialFunctions.wl</code> ; selected real-domain routines	Checked representative finite identities and their domain restrictions. No separate new defect is claimed.
Gamma and Barnes forward/inverse paths	<code>GammaForward.wl</code> ; <code>BarnesForward.wl</code> ; <code>GammaInverse.wl</code> ; <code>GammaInverseOperations.wl</code>	Checked representative coefficient signs, shifts, source-coordinate use and specialized operation boundaries. Candidates overlapping the register were excluded.
Arithmetic and differentiation	<code>SeriesArithmetic.wl</code> ; selected <code>SeriesOperations.wl</code> ; <code>ReciprocalLogOperations.wl</code>	Traced how derivative metadata is consumed and how scale differentiation enters the result. Basis for E2 integration limits.

Area	Files or routines inspected	Result used here
Inverse verification	Selected <code>InverseCertificates.wl</code> ; <code>SourceCoordinates.wl</code> ; <code>CorePerturbation.wl</code> ; Mathics inverse/numerical adapters	Inspected exact interval steps and branch-related boundaries. No new runtime certification is claimed.
Build and documentation	<code>build_standalone.py</code> ; <code>build_user_guide.py</code> ; <code>.gitattributes</code> ; Mathics compatibility guide	Cross-checked selected packaging and documented-environment facts. A proposed line-ending concern was rejected because the repository forces LF.

The finite Lerch source cases are particularly useful controls. When $z = 0$, the function reduces to a^{-s} . For a nonpositive integer s , the summand is polynomial in n/a , and the current provider recognizes a finite expansion. Those observations argue against presenting an alleged generic “nontermination at finite cases” as new evidence: the inspected implementation explicitly handles these cases.[11]

Likewise, the current positive Zeta-tail comparison is valid. For $S > 1$ and first omitted integer m ,

$$m^{-S} \leq \sum_{n=m}^{\infty} n^{-S} \leq m^{-S} \left(1 + \frac{m}{S-1}\right). \quad (1)$$

The upper estimate follows by comparing the decreasing summand with its integral. E2 does not correct this estimate; it proves derivative information not presently attached by default.

For the positive-parameter Lerch subcase used below, the current absolute Taylor estimate is also sound. E1 extracts additional sign and distributional information from the same remainder, instead of treating the valid existing bound as an error. This distinction substantially reduces false positives in a heavily reviewed repository.

3 N1: a source-predicate protection gap in Mathics

3.1 The protection that exists

The Mathics assumption adapter documents a semantic hazard: native membership simplification can replace the realness of a composite expression by an inappropriate condition on an argument. The input adapter therefore protects occurrences of `System‘Element` in syntactic `Assumptions->...` and `Assumptions:>...` option values before releasing a held package computation. It redirects those predicate heads to the package’s Mathics-specific membership implementation.[9, 8]

The key structural test in `mathicsProtectInputAssumptions` is that an `Element` head’s position lies below the right-hand side of one of those option rules. If there are no such rule positions, the held input is returned unchanged. This is a purposeful local guard, not a global replacement of the native membership implementation.

The additional issue is the *other* input location containing a semantic domain predicate: the condition of an inline source `ConditionalExpression`. The current predicate-position selection does not include that location merely because it belongs to the source. An unrelated assumption option does not help: its right-hand-side subtree still does not contain the source predicate.[8]

3.2 Why the difference matters mathematically

Realness of $\log a$ is not equivalent to realness of a . On the principal logarithm, $a = -1$ gives

$$\log(-1) = i\pi \notin \mathbb{R}, \quad -1 \in \mathbb{R}.$$

Thus replacing `Element[Log[a], Reals]` by `Element[a, Reals]` can widen a domain incorrectly. The problem is logical, not a small numerical discrepancy.

The direction of an erroneous simplification is not always widening. For example, $a = \pi/2 + i$ is nonreal but $\sin a = \cosh 1$ is real. Consequently, replacing realness of a sine by realness of its argument can also discard valid parameter values. A generic rule that treats real coefficients or real arguments as a necessary and sufficient characterization of a composite function's real values is not an adequate repair.

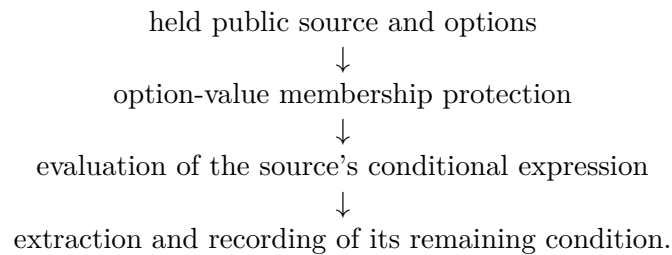
The documented semantics of `ConditionalExpression` make preserving its condition essential: a true condition releases the expression, a false condition gives an undefined result, and conditions can propagate through mathematical expressions. The predicate is part of the source's mathematical meaning, not decorative result metadata.^[18]

3.3 The inspected call path

The public expansion entry is held. In `NativeCompatibility.wl`, an explicit package request is sent to `packageExpansion`; this constructs a held `catch` call around `packageEvaluatedEntry[forwardHeldExpression[f], ...]`. The Mathics `catch` wrapper invokes the option-position guard before evaluating the protected body.^[7, 8]

In the main package, `forwardHeldExpression` has a specific clause for a source `ConditionalExpression`. It passes the condition through into the resulting conditional expression. Later, `forwardPublic` uses `splitApproachInput`; that routine peels outer source conditions and separates parameter-only assumptions from conditions involving the approach variable. That later split cannot restore a predicate that has already been weakened by evaluation.^[6, 10]

The relevant sequence is therefore:



The source-established omission is at the second step. Whether the complete last step produces a wrong answer, an unsupported-domain failure, or some other conservative refusal is a native characterization question.

3.4 A minimal public characterization request

The supplied probe uses the source inline:

```
AsymptoticExpansion[
  ConditionalExpression[x, Element[Log[a], Reals]],
  {x, 0, 2}, Assumptions -> a < 0,
  "Backend" -> "Package"
]
```

For negative real a , the source condition is false. A successful result must not silently assert that this is an ordinary valid expansion of x on that parameter region. A conservative failure or preserved unsatisfied condition is not a wrong answer.

The inline form is important. Assigning the conditional expression to an ordinary variable before calling the package may allow the caller's evaluator to change it before the package obtains control. That is a different boundary, and a package holding attribute cannot recover syntax that never reaches it. N1 concerns syntax that reaches the public held entry intact.

`tests/IncrementalAudit.wl` also includes a structural probe showing whether the option protector leaves the source predicate unchanged, an option-value positive control, a positive- a control and an explicit-condition control. These are unexecuted probes. In particular, the archive does not contain a fabricated native output claiming that the public example returned a particular `GeneralizedSeries`.

3.5 Severity and confidence

The potential impact is high because an incorrectly weakened domain can contaminate a claimed real-branch result. The established fact is narrower: the inspected guard does not protect this source position. The underlying Mathics simplification concern is documented by the repository itself. The public manifestation was not executed here. N1 should therefore be tracked as a *high-priority compatibility characterization*, not as a verified new critical wrong-result regression.

Its nearest historical neighbors concern input assumption handling and conditional-source routing. The distinct question here is predicate preservation at a source-position evaluation boundary, not backend selection, option precedence, ambient-assumption capture, or a new example of a previously recorded sine/assumption failure. The archive's crosswalk records that distinction without reintroducing those older recommendations.[\[2, 3\]](#)

3.6 Proposed repair: protect a parsed source domain, not arbitrary data

The repair belongs before `forwardHeldExpression` releases the source's conditional predicate, and only on the package path that needs the Mathics adapter. The original held request retained for explicit native delegation should remain intact. A source-aware normalization step should identify recognized outer source conditions, retain their logical meaning under hold, and redirect their membership predicates through the conservative Mathics implementation.

A blanket replacement of every `Element` anywhere in the request is not a satisfactory fix. A predicate symbol can occur inside quoted data, an inactive expression, a binder, a conditionally evaluated branch, or an expression whose equality depends on its literal head. Changing such data is itself a semantic change. Similarly, globally altering `System`Element` would make a local package repair an uncontrolled session-wide intervention.

A bounded initial implementation should support the existing outer `ConditionalExpression` source grammar and an explicit boolean condition grammar. It should preserve evaluation count, avoid hoisting a condition out of a scope, and refuse unsupported guarded syntax rather than manufacture a weaker predicate. Conditions produced later by applying a user callable require

their own protected boundary; support for the initial inline form does not prove support for arbitrary dynamically produced source syntax.

The following is a design-level insertion point, not a supplied tested patch:

```
packageExpansion[request_HoldComplete] :=
  releasePackageRequest[
    protectRecognizedMathicsSourceConditions[request]]
```

The new routine must be identity on the Wolfram kernel, identity for predicate-free sources, and identity on unrelated data subtrees. It must not evaluate a delayed option while merely selecting predicate positions. Existing option protection should be retained; the new routine complements it rather than replacing it.

3.7 Acceptance tests for the repair

A native regression should inspect the retained domain and result kind, not only a displayed polynomial. The negative-real logarithm case tests domain widening. The positive-real logarithm case checks that valid input still works. An option-contained membership tests that the existing fix is not lost. A quoted or inactive occurrence checks that data is not rewritten. A delayed predicate with a counter checks evaluation multiplicity. A predicate under an unresolved binder must either remain scoped or be conservatively rejected.

The archive includes the minimal characterization subset, not this entire acceptance matrix implemented and run. The broader matrix is an integration requirement for a production patch. Because native access was unavailable, no change to the repository's own definitions has been applied by this review.

4 E1: a signed mean–variance enclosure for the Lerch remainder

4.1 The exact provider opportunity

The existing Lerch provider expands in the reciprocal of a large positive source argument. It stores coefficients obtained from geometric moments and a Taylor-based absolute remainder bound. For real z of either sign with $|z| < 1$, the absolute-value construction is appropriately general. For $0 < z < 1$ and $s > 0$, however, the summation weights are positive and the Taylor remainder has a definite sign. These extra facts permit a substantially tighter enclosure without changing the expansion coefficients.[11]

The defining series is

$$\Phi(z, s, a) = \sum_{n=0}^{\infty} \frac{z^n}{(a+n)^s} = a^{-s} \sum_{n=0}^{\infty} z^n (1+n/a)^{-s}. \quad (2)$$

For this section, $0 < z < 1$, $s > 0$ and $a > 0$ are real and fixed except for the large- a limit. This is a restricted real subdomain of the standard defining series, not a statement about analytic continuation.[19]

Define moments

$$M_0(z) = \frac{1}{1-z}, \quad M_k(z) = \sum_{n=0}^{\infty} n^k z^n \quad (k \geq 1), \quad (3)$$

and take $N \geq 0$ to be the *first omitted Taylor index*. Thus

$$P_N(z, s, a) = a^{-s} \sum_{k=0}^{N-1} \frac{(-1)^k (s)_k M_k(z)}{k! a^k}, \quad (4)$$

$$E_N = \Phi(z, s, a) - P_N(z, s, a),$$

$$U_N = \frac{(s)_N M_N(z)}{N! a^{s+N}}. \quad (5)$$

The empty polynomial P_0 is zero. This convention avoids confusing a retained-term count with the index used by the remainder formula.

4.2 A probability representation of the signed remainder

For $N \geq 1$, Taylor's integral remainder applied to $h(t) = (1+t)^{-s}$ gives

$$(-1)^N \left(h(t) - \sum_{k=0}^{N-1} \frac{(-1)^k (s)_k}{k!} t^k \right) = \frac{(s)_N t^N}{N!} \int_0^1 N(1-v)^{N-1} (1+vt)^{-s-N} dv. \quad (6)$$

All factors in the integral are nonnegative for $t \geq 0$. Let J_N be a nonnegative integer-valued random variable with weights

$$\Pr(J_N = n) = \frac{n^N z^n}{M_N(z)}.$$

For $N = 0$, use $n^0 = 1$ including $n = 0$. For $N \geq 1$, the mass at zero vanishes. Let V_N be independent of J_N , with density $N(1-v)^{N-1}$ on $[0, 1]$ when $N \geq 1$, and let $V_0 = 1$ deterministically. Set

$$Y_N = \frac{V_N J_N}{a}, \quad p = s + N.$$

Summing (6) against the positive weights in (2), or using the defining series directly when $N = 0$, yields the exact identity

$$\boxed{(-1)^N E_N = U_N \mathbb{E}[(1 + Y_N)^{-p}].} \quad (7)$$

Interchange of the sum and the integral is justified by nonnegativity; the geometric moments are finite for $0 < z < 1$.

The required two moments are explicit:

$$\mu_N = \mathbb{E}[Y_N] = \frac{M_{N+1}(z)}{a(N+1)M_N(z)}, \quad (8)$$

$$\nu_N = \mathbb{E}[Y_N^2] = \frac{2M_{N+2}(z)}{a^2(N+1)(N+2)M_N(z)}, \quad (9)$$

$$\sigma_N^2 = \nu_N - \mu_N^2. \quad (10)$$

These formulas also hold at $N = 0$: the first and second moments of the degenerate variable V_0 are both one. For $N \geq 1$, they follow by integrating the beta density. Their exact polynomial-integral identities are included in the executed Python tests.

4.3 The enclosure theorem

Theorem 4.1 (Signed Lerch enclosure). *Under the positive-parameter assumptions above, define*

$$L_N = U_N(1 + \mu_N)^{-p}, \quad H_N = \min \left\{ U_N, L_N + \frac{U_N p(p+1)}{2} \sigma_N^2 \right\}. \quad (11)$$

Then

$$\boxed{L_N \leq (-1)^N E_N \leq H_N.} \quad (12)$$

Consequently,

$$\Phi(z, s, a) \in \begin{cases} [P_N + L_N, P_N + H_N], & N \text{ even}, \\ [P_N - H_N, P_N - L_N], & N \text{ odd}. \end{cases} \quad (13)$$

Proof. Let $f(y) = (1 + y)^{-p}$ for $y \geq 0$. Then $0 < f(y) \leq 1$ and

$$0 < f''(y) = p(p+1)(1+y)^{-p-2} \leq p(p+1).$$

Convexity gives Jensen's lower bound $\mathbb{E}[f(Y_N)] \geq f(\mu_N)$. Taylor's theorem around μ_N , with the global second-derivative bound on the nonnegative half-line, gives pointwise

$$f(y) \leq f(\mu_N) + f'(\mu_N)(y - \mu_N) + \frac{1}{2}p(p+1)(y - \mu_N)^2.$$

Taking expectations eliminates the linear term and bounds the remainder by $p(p+1)\sigma_N^2/2$. Combining this with $f \leq 1$ and the exact identity (7) proves (12). The parity-dependent interval in (13) then follows from the definition of E_N . $\square$

The simpler Jensen enclosure $L_N \leq (-1)^N E_N \leq U_N$ is already an improvement over a symmetric absolute-value interval. The variance refinement supplies a further order of narrowing and costs two moments beyond the first-omitted index, rather than a new symbolic asymptotic expansion.

4.4 The size of the improvement

With z, s, N fixed, $U_N = O(a^{-s-N})$, $\mu_N = O(a^{-1})$ and $\sigma_N^2 = O(a^{-2})$. The theorem therefore gives

$$H_N - L_N \leq \frac{U_N p(p+1)}{2} \sigma_N^2 = O(a^{-s-N-2}). \quad (14)$$

The current absolute estimate in this positive-parameter subcase has magnitude U_N and gives the symmetric interval $[P_N - U_N, P_N + U_N]$ on its recorded domain. Its width is $2U_N$. Thus the new interval's width relative to that symmetric width is bounded by $p(p+1)\sigma_N^2/4 = O(a^{-2})$.

This is not a claim that the stored N -term polynomial has become an approximation with a two-orders-smaller error. Its error remains of order U_N . The narrower interval is shifted away from the polynomial using the signed remainder information. Confusing enclosure width with truncation error would incorrectly upgrade the expansion itself.

Nor does the result assert convergence as $N \rightarrow \infty$ at fixed a . Increasing an asymptotic order can eventually be unhelpful. The theorem is valid at every finite N in its domain, but an automatic numerical strategy should select an enclosure by its actual width, not assume that a larger order always means a narrower interval.

4.5 Exact-rational implementation

When z, a are rational and s is a positive integer, every endpoint in (13) is rational. The supplied `code/lerch_enclosure.py` computes those endpoints using only Python's standard library and `Fraction`. Approximate floats are rejected rather than silently rationalized. The implementation explicitly restricts the first-omitted index to $0 \leq N \leq 64$; this is a prototype resource boundary, not a mathematical limit.

Moments are computed through Eulerian polynomials. With $A_1(z) = 1$, write

$$M_k(z) = \frac{zA_k(z)}{(1-z)^{k+1}} \quad (k \geq 1).$$

The identity $M_{k+1} = z dM_k/dz$ gives

$$A_{k+1}(z) = (1 + kz)A_k(z) + z(1 - z)A'_k(z). \quad (15)$$

This uses exact integer coefficient updates and rational polynomial evaluation; it does not invoke a special-function simplifier. Building all polynomials through order $N + 2$ requires $O(N^2)$ coefficient updates and $O(N)$ working coefficient storage, excluding the bit complexity of growing integers and rational arithmetic. These are algorithmic operation counts, not measured Wolfram/Mathics speedups.

The independent implementation `code/LerchEnclosure.wl` translates this bounded rational algorithm into Wolfram Language. It does not modify `AsymptoticAnalysis`. It was not executed, so it is a reference candidate rather than a validated drop-in patch. The accompanying `tests/ReferenceWL.wl` contains 108 exact defining-series containment cases for either target kernel.

4.6 Exact execution results

The Python grid covered four values of z , six values of a , three values of s and eight first-omitted indices:

$$\begin{aligned} z &\in \{1/5, 1/2, 3/4, 9/10\}, \\ a &\in \{1/2, 1, 2, 10, 100, 1000\}, \quad s \in \{1, 2, 5\}, \quad N \in \{0, \dots, 7\}. \end{aligned}$$

All 576 cases passed. For each (z, s, a) , an independent rational partial sum of the *convergent defining series* was computed. If K terms are retained, its tail obeys

$$0 \leq \sum_{n=K}^{\infty} \frac{z^n}{(a+n)^s} \leq \frac{z^K}{(a+K)^s(1-z)}. \quad (16)$$

The entire independently certified interval, not merely a decimal estimate, was required to lie inside the new enclosure. The largest K needed on the grid was 512. The reference interval does not reuse Taylor coefficients, Eulerian moments or the Jensen bound, reducing the chance of a shared algebraic mistake in the check.

Cases with $a = 1/2$ validate the extended theorem for $a > 0$; they are not assertions that the current provider advertises its stored bound there. The comparison table below uses $a \geq 1$, within the recorded current bound domain.

For this example $P_3 = 2/a^2 - 4/a^3 + 18/a^4$. At $a = 100$, the implementation obtains $\mu_3 = 3/208$ and $\sigma_3^2 = 28139/135200000$. The exact interval endpoints, which are less readable than the compact formulas, are retained in `evidence/lerch_examples.json`.

a	U_3 for $(z, s) = (1/2, 2)$	New width / $2U_3$	Approximate narrowing factor
10	13/12500	0.156096523669	6.406
100	13/1250000000	0.001560965237	640.6
1000	13/1250000000000000	0.000015609652	64060

Table 3: Exact-rational calculations, with ratios rounded only for display. The factor compares interval widths, not function-evaluation runtime.

4.7 Boundary cases and integration constraints

At $z = 0$ the source is exactly a^{-s} . For $N = 0$, the theorem reduces to that exact value; for $N > 0$, $U_N = 0$ and the moment quotient defining μ_N must not be evaluated. The implementation handles this branch before division. Nonpositive s , negative z , complex parameters and $z = 1$ are outside the new signed theorem. They must retain the existing valid path or be refused by this helper; positivity must not be inferred from an unresolved symbolic comparison.

As $z \rightarrow 1^-$, the moments grow and the constants need not be uniform. The extension carries the fixed-parameter scope of the provider. It is not a solution of a simultaneous $a \rightarrow \infty$, $z \rightarrow 1$ transition problem.

A package integration can leave the current coefficient rows, first-omitted index and absolute-bound contract unchanged, adding a signed interval contract on the restricted domain. The interval's lower endpoint is a bound for $(-1)^N E_N$, not always a lower bound for E_N itself. Metadata must state that sign convention explicitly. The safer schema is a two-endpoint *remainder interval* after applying parity, accompanied by its conditions and first-omitted index.

The review does not attach an inverse-function `NumericCertificate` flag to this forward theorem. Exact rational forward enclosures and the package's inverse certificate contract are different objects. General transport of quantitative bounds through arbitrary arithmetic is also not claimed or reproposed here; that topic already belongs to the exclusion baseline.

5 E2: differentiable Zeta Dirichlet tails

5.1 The current conservative contract

The Zeta provider accepts an affine source argument tending to positive infinity on the admitted real approach. It uses the positive coordinate $w = \exp(-S)$ and rows with exponents $\log n$. The common Dirichlet constructor records `RemainderDerivativeOrder->0` whenever the retained result has a nonzero finite tail. This is conservative: a magnitude bound alone does not justify differentiation.[11]

The general derivative routine checks this field and raises `UnprovedRemainderDerivative` when a requested derivative exceeds the available order, unless a sufficient derivative contract is explicitly declared. Thus the repository already offers an escape hatch for a user who has supplied the proof. E2's opportunity is to establish and record that proof automatically for this specific provider, rather than introduce a new generic differentiation facility.[12]

5.2 The tail in its stored coordinate

For the first omitted integer $m \geq 1$, define

$$R_m(w) = \sum_{n=m}^{\infty} w^{\log n}, \quad 0 < w < e^{-1}. \quad (17)$$

This is the standard Zeta defining tail after $S = -\log w > 1$. [20] For every nonnegative integer r , the formal term derivative is

$$\frac{d^r}{dw^r} w^{\log n} = (\log n)_r w^{\log n - r}, \quad (18)$$

where $(t)_r = t(t-1) \cdots (t-r+1)$ and the empty product is one.

The required question is not whether one can formally write (18), but whether it is a derivative of the summed tail and obeys the order advertised by the representation.

Theorem 5.1 (All-orders Dirichlet tail bound). *Fix $S_0 > 1$, $m \geq 1$ and $r \geq 0$. For $0 < w \leq e^{-S_0}$,*

$$|R_m^{(r)}(w)| \leq C_{m,r,S_0} w^{\log m - r}, \quad (19)$$

where the finite constant

$$C_{m,r,S_0} = (\log m + r)^r + m \sum_{j=0}^r \binom{r}{j} \frac{(\log m + r)^{r-j} j!}{(S_0 - 1)^{j+1}} \quad (20)$$

is valid, with zeroth powers interpreted as one.

Proof. Write $\lambda_n = \log n$. For $\lambda_n \geq 0$, $|(\lambda_n)_r| \leq (\lambda_n + r)^r$. Moreover,

$$w^{\lambda_n - \log m} \leq e^{-S_0(\lambda_n - \log m)} = (n/m)^{-S_0}.$$

Consequently, the absolute sum of the differentiated terms is bounded by

$$w^{\log m - r} m^{S_0} \sum_{n=m}^{\infty} (\log n + r)^r n^{-S_0}.$$

The function $g(t) = (\log t + r)^r t^{-S_0}$ is decreasing for $t \geq 1$. For $r > 0$, its logarithmic derivative multiplied by t is $r/(\log t + r) - S_0 \leq 1 - S_0 < 0$; for $r = 0$ the conclusion is immediate. Hence the sum is at most $g(m) + \int_m^{\infty} g(t) dt$.

Substitute $t = me^v$ and expand the integer power:

$$\int_m^{\infty} g(t) dt = m^{1-S_0} \sum_{j=0}^r \binom{r}{j} (\log m + r)^{r-j} \frac{j!}{(S_0 - 1)^{j+1}}.$$

Multiplication by m^{S_0} gives (20). On every compact subinterval bounded away from $w = 0$, these summable majorants justify termwise differentiation to any fixed order. Since the order was arbitrary, (19) follows for every r . $\square$

The resulting remainder belongs to the differentiated power class

$$R_m^{(r)}(w) = O(w^{\log m - r}),$$

with logarithmic degree zero. No analyticity at $w = 0$ is asserted. The exponents are generally nonintegral, and the endpoint itself is not a point at which an ordinary convergent Taylor expansion is being claimed.

5.3 The original source variable

For the admitted affine argument $S(x) = \alpha x + \beta$, $\alpha \neq 0$, differentiation in the source variable is even simpler:

$$\frac{d^r}{dx^r} R_m(e^{-S(x)}) = (-\alpha)^r \sum_{n=m}^{\infty} (\log n)^r n^{-S(x)}.$$

Using the same majorant gives the explicit bound

$$\left| \frac{d^r}{dx^r} R_m(e^{-S(x)}) \right| \leq |\alpha|^r C_{m,r,S_0} m^{-S(x)} \quad (S(x) \geq S_0). \quad (21)$$

Thus source-variable differentiation does not introduce a new exponential decay rate. The chain-rule factors compensate the w^{-r} factor that appears when differentiating with respect to the stored coordinate.

For instance, three retained Dirichlet terms of $\zeta(x)$ give

$$1 + 2^{-x} + 3^{-x} + R_4(e^{-x}).$$

Their first source derivative has finite part

$$-\log(2)2^{-x} - \log(3)3^{-x}$$

and a tail of order 4^{-x} . This is a mathematically derived target for the native probe, not an observed output of the package in this environment.

5.4 A narrow integration

The common constructor is also used by the Lerch provider, whose admitted source argument has a different scope. It would be unjustified to change the common constructor's finite-tail default from zero to infinity indiscriminately. The Zeta proof should be attached only to the path satisfying its actual affine, positive-large-argument admission conditions.

One possible change is an optional internal derivative-order argument defaulting to zero. The existing Zeta call supplies infinity, while the Lerch call retains the old default. The representation must receive that argument *before* `seriesMake` builds the result. Adding a disconnected top-level annotation after construction is not equivalent to updating the field consumed by the derivative engine.

The public workaround already available, justified for this Zeta path by Theorem 5.1, is conceptually:

```
SeriesDifferentiate[zetaSeries, 1,
  "RemainderDerivativeOrder" -> Infinity]
```

The archive includes both the default-contract and explicit-contract characterization calls. Neither was executed. The latter is a proof-backed use of an existing declaration mechanism; it must not be generalized to an arbitrary object merely because its finite displayed expression can be differentiated.

An integration test should include affine arguments with either source orientation, the case where differentiation removes the retained constant term, several derivative orders, and agreement of the resulting expression with the explicit Dirichlet derivative sum. It should inspect the representation-level contract after the operation, not only the top-level printed expression. Bound (21) supplies an independent mathematical oracle for the tail class.

6 Validation, portability and artifact status

6.1 What was actually executed

The executed component is the independent Python exact-rational implementation. Seven test groups passed: known geometric moments, the $z = 0$ exact case, input-domain rejection, the 576-case containment grid, beta moment identities, the finite Laplace-sum recurrence used in the Zeta bound, and the second-order normalized enclosure-width behavior. The log and complete grid records are bundled.

The beta tests integrate $N(1 - v)^{N-1}v^k$ by expanding the polynomial for $1 \leq N \leq 16$ and $k = 1, 2$, then compare exact rational values with the moment formulas. The Zeta auxiliary tests check the recurrence for

$$J_r(A, c) = \sum_{j=0}^r \binom{r}{j} \frac{A^{r-j} j!}{c^{j+1}}, \quad cJ_r - rJ_{r-1} = A^r,$$

on twelve exact rational (A, c) pairs through $r = 12$. These checks support the implemented algebra; the infinite-tail derivative theorem rests on the proof, not on the finite grid.

The environment reported Python 3.13.5. The test log contains the measured execution time for this reference suite. That time must not be compared with the package's runtime: no package invocation, kernel startup, native simplification or Wolfram/Mathics memory measurement occurred in the run.

6.2 What was not executed

Neither Wolfram 15 nor Mathics was installed in the accessible container. The attempted connected Wolfram evaluator returned an error rather than a kernel result. No claim about a successful native session, kernel version, package test count, or backend parity is inferred from that failure.

The Wolfram Language rational implementation, its 108-case check file and the public package characterization probes are therefore explicitly labelled *unexecuted*. The package probes emit raw result summaries rather than declaring a public regression confirmed from an assumed error tag. Their environmental record includes the actual kernel version when the user runs them.

6.3 Portability implications of the proposed changes

N1 must be assessed on Mathics because its concern is a Mathics-specific evaluation boundary. The Wolfram kernel is the control for preserving the documented predicate semantics, not evidence that the Mathics guard works. The proposed repair must leave native delegation requests untouched and must not mutate `System'` definitions.

E1's exact-rational implementation deliberately avoids native `PolyLog` evaluation and assumption-based simplification. Its restricted input domain makes failures explicit: exact rational z, a , positive integer s , bounded nonnegative integer N . That design lowers the amount of backend-specific symbolic behavior required by this particular helper, but actual Mathics behavior of the Wolfram translation remains untested. It is not a claim of general package compatibility.

E2's proof is backend-independent, but representation construction and later differentiation are not. A correct theorem authorizes a particular contract value only when the provider's admission conditions and source function match the theorem. The proposed integration preserves this separation: source recognition, contract assignment and public operation tests must all agree.

6.4 Source-grounding and coverage limitations

The repository reads covered selected modules and substantial contiguous ranges, not every executable line and every workflow. No automated whole-tree static analyzer was run against a local clone. Deduplication principally used the maintained review registers and intake indexes, rather than an exhaustive text comparison against all historical article bytes.

These limitations bound the conclusion: the article is an incremental technical review with concrete proof and implementation work, not a complete formal verification or an exhaustive inventory of all remaining defects. The supplied evidence manifest marks native execution as unavailable and distinguishes source-established omissions from mathematical enhancements. It also records that the repository itself was not edited.

7 Integration priorities and development recommendations

7.1 Priority 1: characterize and close N1's exact boundary

Run the inline negative-logarithm predicate case and its controls on the targeted Mathics version and on Wolfram 15. Preserve the raw domain-bearing result. A successful expansion whose conditions have been incorrectly widened is a correctness defect; a conservative refusal demonstrates a capability gap instead. This decision must precede assigning a confirmed wrong-result label.

When the unprotected evaluation path is confirmed, place the repair at the held source boundary on the package route. Make its supported source-condition grammar explicit. The acceptance target is predicate equivalence on the original source domain, not merely agreement of a polynomial finite part. Do not broaden the patch into session-wide membership rewriting.

7.2 Priority 2: add the positive-parameter Lerch enclosure

The exact rational helper is ready as an independently tested reference, not as an already merged package feature. A practical first integration can use the restricted rational/positive-integer domain of the supplied implementation. It needs the current provider's first-omitted Taylor index, moments through $N + 2$, the positivity conditions and the parity convention. Its result should expose a remainder interval whose interpretation is unambiguous.

A later extension to positive noninteger real s is mathematically justified by Theorem 4.1, but numerical enclosure of noninteger powers must use an appropriate outward-bounding implementation. Merely evaluating those endpoints at high precision would not preserve the exact-rational guarantee of the reference implementation. Negative z and nonpositive s should keep the existing general absolute-bound route.

The immediate performance opportunity is local and testable: compare moment generation and endpoint construction with the present positive-parameter coefficient/bound path at equal requested order. The article provides operation counts, not an unmeasured speedup. The expected gain in interval width is already proved and tested; runtime gain remains an empirical question.

7.3 Priority 3: make the admitted Zeta derivative contract automatic

Thread the derivative-order contract through the existing constructor before the representation is finalized, selecting infinity only for the proven Zeta path. Confirm that both a direct first

derivative and repeated derivatives preserve the intended tail class and branch conditions. The existing explicit declaration is a useful control.

The finite constant in Theorem 5.1 can also be recorded as proof documentation for any future quantitative derivative-bound interface, but the automatic derivative capability does not require evaluating that potentially large bound on every call. The theorem and its admission conditions justify the asymptotic contract independently of numerical materialization of the constant.

7.4 Bounded future work, rather than a repeated roadmap

The natural continuation of E1 is to use additional moments to tighten the expectation enclosure while retaining a finite proof and an explicit cost. For example, a higher-order convexity argument would have to control derivative signs and remainder terms; simply fitting a distribution to the first moments would not be a certificate. The present mean–variance formula is attractive because the global second-derivative bound is elementary and does not require solving a moment problem.

The natural continuation of E2 is a provider-specific library of convergent-tail derivative proofs, not a global permission to differentiate arbitrary big- O objects. A new provider should supply its own summable majorant and coordinate conditions. This article establishes that obligation for the current Zeta path only.

Neither continuation repeats the existing general multiscale, global arithmetic-transport, certification-framework or release-process roadmap. They are bounded developments of the new constructions actually proved here.

8 Conclusion

A heavily reviewed symbolic repository should not be evaluated by the number of familiar issues a new report can restate. At the pinned revision, the strongest incremental contributions from this inspection are a sharply located Mathics source-predicate protection risk and two concrete ways to strengthen existing special-function providers.

N1 separates a proven omission in a held-syntax guard from an unexecuted public failure prediction. E1 gives an exact signed remainder representation and a mean–variance enclosure with relative interval-width improvement of order a^{-2} ; its rational implementation passed all 576 independent containment cases. E2 proves all-orders derivative bounds for the admitted Zeta tail and explains how to attach that information without overgranting capability to a shared constructor’s other clients.

The archive supplies reusable code, raw reference evidence and native characterization probes. It does not assert dual-runtime conformance, claim globally novel mathematics, apply untested monkey patches, or reissue the repository’s already recorded findings as new results.

A Running the supplied artifacts

From the extracted archive directory, run the independent reference checks with

```
python tests/test_reference.py
```

The Python implementation uses the standard library only. The command updates the grid evidence JSON. Redirect standard output and standard error to a new log when retaining another run. Existing packaged evidence describes the run performed during this review, not a future user's environment.

For the Wolfram Language reference, evaluate these two commands in a kernel:

```
Get["/absolute/path/to/code/LerchEnclosure.wl"];
Get["/absolute/path/to/tests/ReferenceWL.wl"];
```

The second file performs 108 exact checks and prints the actual kernel version and failures. These are unexecuted in the delivered review.

For the package characterization, first load the *pinned* repository in a fresh target-kernel session, then evaluate

```
Get["/absolute/path/to/tests/IncrementalAudit.wl"];
AsymptoticAudit[AsymptoticAuditResults
```

The script does not fetch, install, patch or update the repository. It expects the package already loaded, and it does not treat absence of an observed exception as proof that every invariant passed. Read the N1 result's conditions and compare the E2 expression with the mathematical derivative in Section 5.

To rebuild the article:

```
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
```

Two passes resolve the table of contents, cross-references and bibliography. The archive includes the built PDF; a LaTeX installation is needed only to rebuild it.

B Artifact inventory

File	Meaning
article.tex, article.pdf	This article's editable source and rendered version.
code/lerch_enclosure.py	Executed exact-rational reference implementation, not repository code.
code/LerchEnclosure.wl	Unexecuted Wolfram Language translation of the restricted rational helper.
tests/test_reference.py	Seven executed Python test groups, including 576 containment cases.
tests/ReferenceWL.wl	108 unexecuted Wolfram/Mathics reference checks.
tests/IncrementalAudit.wl	Unexecuted N1 and E2 package characterization probes.
evidence/python_test_log.txt	Actual reference-suite console output.

File	Meaning
evidence/python_reference_results.json	Complete grid, exact-containment outcomes and environment.
evidence/lerch_examples.json	Exact rational sample endpoints and display ratios.
evidence/review_manifest.json	Snapshot, scope, execution classification and source inventory.
evidence/novelty_crosswalk.md	Nearest earlier review categories and why the incremental mechanisms differ.
README.md	Usage, limits and the distinction between executed and unexecuted artifacts.

References

- [1] Asymptotic repository. [external-reports/code-review/README.md](#). Review index and retirement policy. Pinned revision stated on the title page.
- [2] Asymptotic repository. [docs/development/CODE_REVIEW_STATUS.md](#). Central review status and mechanism register.
- [3] Asymptotic repository. [docs/development/WAVE_3_INTAKE.md](#). Wave-three intake and deduplication record.
- [4] Asymptotic repository. [docs/development/WAVE_4_INTAKE.md](#). Wave-four intake and deduplication record; also [external-reports/code-review/wave-4/README.md](#).
- [5] Asymptotic repository. [external-reports/code-review/wave-5/README.md](#). Latest retained/retired package index consulted.
- [6] Asymptotic repository. [src/Kernel/AsymptoticAnalysis.wl](#). Main package entry and forward constructions; selected ranges.
- [7] Asymptotic repository. [src/Kernel/NativeCompatibility.wl](#). Held package/native dispatch.
- [8] Asymptotic repository. [src/Kernel/MathicsInputAssumptions.wl](#). Input predicate protection and the held wrapper.
- [9] Asymptotic repository. [src/Kernel/MathicsAssumptions.wl](#). Conservative Mathics assumption and membership handling.
- [10] Asymptotic repository. [src/Kernel/InverseFunctionExpressions.wl](#); [src/Kernel/InverseFunctionSyntax.wl](#). Source-condition extraction, forward dispatch and callable parsing.
- [11] Asymptotic repository. [src/Kernel/DirichletSpecialFunctions.wl](#). Zeta and Lerch providers and their contracts.
- [12] Asymptotic repository. [src/Kernel/SeriesOperations.wl](#). Arithmetic, differentiation and refinement; selected ranges.
- [13] Asymptotic repository. [src/Kernel/MathicsTaylor.wl](#); [src/Kernel/MathicsNumerical.wl](#); [src/Kernel/MathicsInverseBranches.wl](#); [src/Kernel/MathicsCompatibility.wl](#). Additional adapters inspected.
- [14] Asymptotic repository. [src/Kernel/SpecialFunctionIdentities.wl](#); [src/Kernel/ParameterizedSpecialFunctions.wl](#); [src/Kernel/SpecialFunctionRealDomain.wl](#); [src/Kernel/GammaForward.wl](#); [src/Kernel/BarnesForward.wl](#). Representative identity, coefficient and domain paths inspected.
- [15] Asymptotic repository. [src/Kernel/GammaInverse.wl](#); [src/Kernel/GammaInverseOperations.wl](#); [src/Kernel/SourceCoordinates.wl](#); [src/Kernel/CorePerturbation.wl](#); [src/Kernel/InverseCertificates.wl](#). Selected inverse and certificate paths inspected.
- [16] Asymptotic repository. [src/Kernel/SeriesArithmetic.wl](#); [src/Kernel/ReciprocalLogOperations.wl](#). Selected arithmetic and scale-operation paths inspected.
- [17] Asymptotic repository. [validation/build_standalone.py](#); [validation/build_user_guide.py](#); [.gitattributes](#); [docs/Mathics/COMPATIBILITY.md](#). Build and documented compatibility context.
- [18] Wolfram Research. *ConditionalExpression*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/ConditionalExpression.html>. Consulted 10 September 2026.
- [19] National Institute of Standards and Technology. *Digital Library of Mathematical Functions*, §25.14, especially Eq. 25.14.1 (defining series). <https://dlmf.nist.gov/25.14>. Consulted 10 September 2026.
- [20] National Institute of Standards and Technology. *Digital Library of Mathematical Functions*, §25.2, especially Eq. 25.2.1 (defining series). <https://dlmf.nist.gov/25.2>. Consulted 10 September 2026.